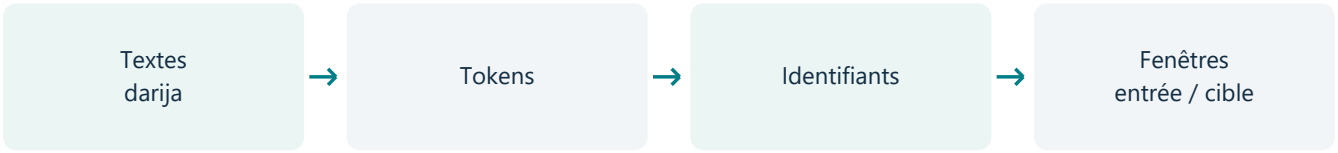
Créer un tokenizer

Cours pratique en français / Darija marocaine / Niveau débutant Python

Du texte darija aux nombres du modèle

Construire un vocabulaire, comprendre BPE et préparer les entrées de notre futur GPT.

Ce cours accompagne notre étude du chapitre 2 de Build a Large Language Model (From Scratch). Nous utilisons les **datasets d'AtlasIA** et apprenons notre propre vocabulaire. La bibliothèque Tokenizers exécute l'algorithme BPE ; les fusions sont apprises sur notre corpus.



Ce que tu sauras faire

Comprendre / p. 2-4	Différencier token, identifiant et embedding ; coder un premier tokenizer.
Construire / p. 5-8	Préparer le corpus, apprendre BPE, encoder, décoder et sauvegarder.
Vérifier / p. 9-12	Mesurer la fragmentation, créer les fenêtres et résoudre les exercices.
Pratiquer / p. 13	Exécuter le script compagnon et retrouver les sources officielles.

> **Résultat attendu :** un fichier `tokenizer_darija.json` contenant les règles, les tokens spéciaux, le vocabulaire et les fusions. Ce fichier sera réutilisé avec le futur modèle.

Pré-requis : listes, dictionnaires et fonctions Python. Lis les pages 2 à 6 avant de lancer le script. Le premier tokenizer reste volontairement simple ; BPE constitue l'atelier principal.

Tokens, vocabulaire, identifiants

La base : découper le texte et nommer chaque morceau avec un nombre.

Un **token** est une unité choisie par le tokenizer : mot, fragment, caractère ou représentation d'octets. Le **vocabulaire** associe chaque unité à un entier. Le même tokenizer doit produire les mêmes identifiants à l'entraînement et à l'utilisation.

EXEMPLE ILLUSTRATIF - VOCABULAIRE CHOISI À LA MAIN

```
texte = "salam, labas?"
tokens = ["salam", ",", "labas", "?"]
vocab = {"[UNK]": 0, "salam": 1, ",": 2, "labas": 3, "?": 4}
ids = [vocab[token] for token in tokens]
print(ids)  # [1, 2, 3, 4]
```

Ces identifiants sont des étiquettes. Le token 4 n'est ni deux fois plus important que le token 2, ni nécessairement proche de lui par le sens. Les sorties exactes de notre futur BPE dépendront du corpus et des paramètres d'apprentissage.

Trois objets à distinguer

Objet	Rôle
Token	Un morceau de texte reconnu par les règles du tokenizer.
Identifiant	Un entier qui indexe une entrée du vocabulaire.
Embedding	Un vecteur de nombres associé à cet identifiant dans le réseau de neurones.

Plusieurs écritures pour la darija

سلام، لاباس؟

Exemple pédagogique en écriture arabe : une salutation.

AUTRES EXEMPLES PÉDAGOGIQUES

```
"salam, labas?"          # Arabizi / alphabet latin
"ghadi nji demain."      # Melange darija-francais
```

> Un tokenizer peut représenter ces écritures sans savoir qu'elles ont parfois un sens proche. L'apprentissage du sens et du contexte viendra avec le modèle.

Coder un premier tokenizer

Comprendre encode, decode et le dictionnaire avant d'aborder BPE.

Cette version découpe les suites de caractères de mot et isole la ponctuation. Elle trie les morceaux rencontrés pour créer un vocabulaire stable. Le token `[UNK]`, d'identifiant 0, remplace une unité inconnue.

PYTHON

```
import re
import unicodedata

def normalize(text):
    return unicodedata.normalize("NFC", text)

class WordTokenizer:
    """Version pedagogique : le decodage ne preserve pas les espaces."""

    def __init__(self, texts):
        pieces = {
            piece for text in texts
            for piece in re.findall(r"\w+|[^\w\s]", normalize(text))
        }
        self.vocab = {"[UNK]": 0}
        self.vocab.update({s: i + 1 for i, s in enumerate(sorted(pieces))})
        self.inverse = {i: s for s, i in self.vocab.items()}

    def encode(self, text):
        pieces = re.findall(r"\w+|[^\w\s]", normalize(text))
        return [self.vocab.get(s, 0) for s in pieces]

    def decode(self, ids):
        return " ".join(self.inverse[i] for i in ids)
```

ESSAI LOCAL - DONNÉES INVENTÉES POUR COMPRENDRE LE CODE

```
toy = WordTokenizer(["salam, labas?"])
ids = toy.encode("salam, labas?")
print(toy.vocab)
print(ids)              # [4, 1, 3, 2]
print(toy.decode(ids))  # salam , labas ?
```

`re.findall` extrait les morceaux ; `sorted` fixe leur ordre ; `dict.get` gère les inconnus. Le dictionnaire inverse sert à décoder. Le code du script compagnon utilise la même classe.

> Le décodage par mots réinsère un espace entre les unités : il ne restitue pas exactement la ponctuation ni les espaces d'origine. Ce comportement est une limite de cette première version.

Adapter le traitement à la darija

Préserver l'information avant de chercher à réduire la taille du vocabulaire.

1. Les variantes ne sont pas des erreurs de code

Un même usage peut avoir plusieurs graphies. En Arabizi, les chiffres comme 3 et 7 font partie des conventions d'écriture. Garder les caractères latins, arabes, les chiffres, la ponctuation et les accents évite de perdre une partie du texte.

الله يعطيك الصحة

Autres graphies possibles : llah y3tik se7a / lah y3tik saha. Le tokenizer les voit comme des chaînes différentes.

2. Choisir une normalisation explicite

Nous choisissons **NFC**, une normalisation Unicode conservative. Elle rapproche certaines représentations techniquement équivalentes. Elle ne traduit pas, ne corrige pas l'orthographe et ne transforme pas l'Arabizi en arabe.

MÊME APPARENCE, REPRÉSENTATIONS UNICODE DIFFÉRENTES

```
import unicodedata
text = "cafe\u0301"  # e suivi d'un accent combinant
clean = unicodedata.normalize("NFC", text)
assert clean == "caf\u00e9"
assert len(text) == 5 and len(clean) == 4
```

NFC ne remplace pas automatiquement les formes arabes de présentation par des lettres standard. Une normalisation plus forte comme NFKC peut le faire pour certains caractères ; elle demande une vérification sur le corpus avant d'être adoptée.

3. Comprendre le problème des inconnus

PYTHON

```
toy = WordTokenizer(["salam labas"])
print(toy.encode("salam khouya"))  # [2, 0]
print(toy.decode([2, 0]))          # salam [UNK]
```

> Une fois remplacé par [UNK], le mot inconnu est perdu. BPE réduit ce problème en composant des unités plus petites. La variante sur octets que nous allons utiliser couvre les 256 valeurs d'octets dès le départ.

Préparer les données AtlasIA

Séparer les documents avant d'apprendre le vocabulaire ou les fusions.

Nous partons du dataset public **atlasia/darija_english**, fichier `atlasia_stories.csv`, et gardons uniquement la colonne `darija`. La page 13 indique où télécharger le CSV. Le jeu affiche une licence CC BY-NC 4.0 [1].

On sélectionne au plus 1 000 documents pour l'atelier. On applique NFC, retire les blancs aux bords des documents et enlève les doublons exacts. Ce petit échantillon sert à comprendre le pipeline ; son début peut être biaisé par l'ordre du fichier.

PYTHON

```
import csv
import random
from pathlib import Path

def read_corpus(csv_path, limit=1000):
    if limit < 2:
        raise ValueError("Il faut au moins deux documents.")
    documents, seen = [], set()
    with Path(csv_path).open(encoding="utf-8-sig", newline="") as stream:
        rows = csv.DictReader(stream)
        if "darija" not in (rows.fieldnames or []):
            raise ValueError("Colonne 'darija' absente : verifier le CSV AtlasIA.")
        for row in rows:
            text = normalize(row.get("darija") or "").strip()
            if not text or text in seen:
                continue
            seen.add(text)
            documents.append(text)
            if len(documents) >= limit:
                break
    if len(documents) < 2:
        raise ValueError("Le CSV contient moins de deux textes distincts.")
    return documents

def split_corpus(documents, seed=42):
    documents = list(documents)
    random.Random(seed).shuffle(documents)
    cut = max(1, min(len(documents) - 1, int(0.9 * len(documents))))
    return documents[:cut], documents[cut:]
```

Après cette préparation : `train_texts, valid_texts = split_corpus(read_corpus('atlasia_stories.csv'))`. Le tokenizer apprend seulement sur `train_texts`. Les textes apparentés et quasi-doublons demanderaient un regroupement supplémentaire pour une évaluation sérieuse.

Comprendre BPE avec un exemple

BPE = Byte Pair Encoding : apprendre des fusions fréquentes.

Commençons avec un **exemple simplifié sur caractères** : le corpus contient deux fois abab et une fois ab. On cherche les paires adjacentes dans chaque séquence, sans relier deux documents.

UNE PREMIÈRE FUSION

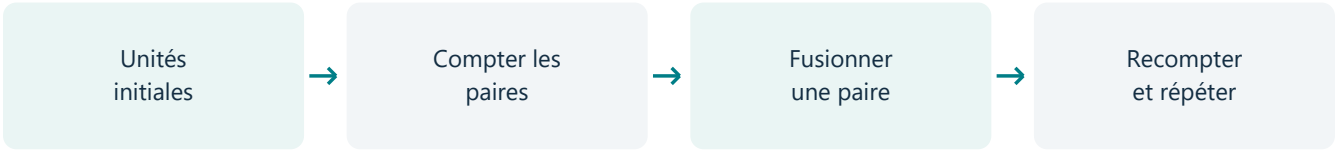
```
Avant : [a, b, a, b]   [a, b, a, b]   [a, b]

Paire (a, b) : 5 occurrences
Paire (b, a) : 2 occurrences

Fusion choisie : a + b -> ab

Apres : [ab, ab]       [ab, ab]       [ab]
```

On recompte les paires après chaque fusion. Ici, `(ab, ab)` apparaît ensuite deux fois. En la fusionnant, on ajoute l'unité abab. Les unités précédentes restent disponibles dans le vocabulaire.



Apprendre et encoder sont deux opérations distinctes

Opération	Ce qui se passe
Apprentissage BPE	Compter les paires dans le corpus d'entraînement et enregistrer l'ordre des fusions.
Encodage	Appliquer les règles déjà apprises à un nouveau texte. On ne réapprend pas le vocabulaire.

La variante sur octets

Le code de la page suivante utilise **ByteLevel BPE**. Le texte Unicode devient une suite d'octets UTF-8, ensuite regroupés par des fusions. Une lettre arabe peut occuper plusieurs octets : octet, caractère et token sont trois unités différentes [2].

> BPE apprend des fréquences et des règles de fusion. Il n'entraîne pas un réseau de neurones et ne donne pas de sens aux tokens. Les fusions ne sont pas obligées de correspondre à des mots complets.

Apprendre notre tokenizer BPE

Le vocabulaire et les fusions sont créés à partir du corpus d'entraînement.

La bibliothèque `tokenizers` fournit l'algorithme. Nous initialisons un BPE vide et lui présentons nos textes. Pour reproduire l'atelier testé, installe la version 0.22.1 ; cette version est un choix de reproductibilité.

TERMINAL

```
python -m pip install tokenizers==0.22.1
```

PYTHON

```
from tokenizers import Tokenizer, decoders, models, normalizers
from tokenizers import pre_tokenizers, trainers

SPECIALS = ["[UNK]", "[PAD]", "[BOS]", "[EOS]"]

def train_bpe(train_texts, vocab_size=2000):
    if vocab_size < 256 + len(SPECIALS):
        raise ValueError("La taille doit couvrir 256 octets et les tokens speciaux.")
    tokenizer = Tokenizer(models.BPE(unk_token="[UNK]"))
    tokenizer.normalizer = normalizers.NFC()
    tokenizer.pre_tokenizer = pre_tokenizers.ByteLevel(add_prefix_space=False)
    tokenizer.decoder = decoders.ByteLevel()
    trainer = trainers.BpeTrainer(
        vocab_size=vocab_size,
        min_frequency=2,
        initial_alphabet=pre_tokenizers.ByteLevel.alphabet(),
        special_tokens=SPECIALS,
        show_progress=False,
    )
    tokenizer.train_from_iterator(train_texts, trainer=trainer)
    tokenizer.encode_special_tokens = True
    return tokenizer

tokenizer = train_bpe(train_texts, vocab_size=2000)
```

ByteLevel garde une représentation de tous les octets, y compris ceux absents de l'échantillon. Son décodeur effectue l'opération inverse. `add_prefix_space=False` évite l'ajout artificiel d'un espace au début [2].

`vocab_size` inclut les unités initiales et les tokens spéciaux. C'est une cible : le vocabulaire réel peut être plus petit si les paires restantes ne satisfont plus `min_frequency`. Toujours lire `tokenizer.get_vocab_size()` [3].

Encoder, décoder, sauvegarder

Un tokenizer utilisable doit respecter des invariants vérifiables.

PYTHON

```
text = "salam, labas?"
encoded = tokenizer.encode(text, add_special_tokens=False)
print(encoded.tokens)  # Unites internes du ByteLevel
print(encoded.ids)     # Identifiants du vocabulaire appris
restored = tokenizer.decode(encoded.ids, skip_special_tokens=False)
assert restored == normalize(text)
```

Les chaînes de `encoded.tokens` peuvent sembler étranges : ByteLevel affiche une représentation de ses octets. Pour vérifier le texte, décode la **séquence entière**. Un token isolé peut contenir seulement une partie des octets d'une lettre arabe.

Tester plusieurs formes d'écriture

Teste aussi les phrases arabes des pages 2 et 4, les espaces répétés, les accents et les emoji.

PYTHON

```
def check_roundtrip(tokenizer, samples):
    for text in samples:
        ids = tokenizer.encode(text, add_special_tokens=False).ids
        restored = tokenizer.decode(ids, skip_special_tokens=False)
        assert restored == normalize(text), (text, restored)
        assert tokenizer.token_to_id("[UNK]") not in ids

samples = ["salam  labas?", "cafe\u0301", "\U0001f642"]
samples += valid_texts[:20]
check_roundtrip(tokenizer, samples)
```

Enregistrer tout le pipeline

PYTHON

```
tokenizer.save("tokenizer_darija.json")
loaded = Tokenizer.from_file("tokenizer_darija.json")
loaded.encode_special_tokens = True
assert loaded.encode(text).ids == tokenizer.encode(text).ids
```

> Rétablir `encode_special_tokens=True` après rechargement : ce drapeau garde les chaînes comme [EOS] dans le texte ordinaire. Il n'est pas enregistré dans le JSON. Les marqueurs réels seront ajoutés par leur identifiant.

Évaluer les choix du tokenizer

Mesurer sur la validation, en gardant le même texte et la même normalisation.

Un tokenizer peut décoder correctement et fragmenter beaucoup trop le texte. Compare donc plusieurs tailles de vocabulaire apprises sur **les mêmes documents d'entraînement**, puis mesure leur comportement sur la même validation.

PYTHON

```
def validation_stats(tokenizer, texts):
    encodings = [tokenizer.encode(t, add_special_tokens=False) for t in texts]
    n_tokens = sum(len(e.ids) for e in encodings)
    n_chars = sum(len(normalize(t)) for t in texts)
    unk = tokenizer.token_to_id("[UNK]")
    n_unk = sum(e.ids.count(unk) for e in encodings)
    return {
        "validation_documents": len(texts),
        "validation_tokens": n_tokens,
        "tokens_per_100_characters": round(100 * n_tokens / max(1, n_chars), 2),
        "unknown_tokens": n_unk,
    }

print(validation_stats(tokenizer, valid_texts))
```

La mesure **tokens pour 100 caractères** est un indicateur pratique, pas une note de compréhension. Python compte ici les points de code Unicode, qui ne sont pas toujours des caractères visibles. Compare également l'arabe et l'Arabizi séparément si tu disposes de groupes annotés.

Observation	Interprétation
Vocabulaire plus petit	Moins de lignes dans la future table d'embeddings ; texte souvent plus fragmenté.
Vocabulaire plus grand	Possibilité de séquences plus courtes ; davantage de paramètres dans les couches liées au vocabulaire.
Zéro [UNK]	Vérification de couverture attendue avec les 256 octets ; ne mesure pas la qualité linguistique.

> L'unité de contexte du GPT sera le token. Un même texte qui demande davantage de tokens consommera davantage de places dans la fenêtre de contexte.

Tokens spéciaux : UNK représente une unité inconnue ; PAD complète une séquence ; BOS marque un début ; EOS marque une fin. Les réserver ne les insère pas automatiquement. Dans notre atelier, seul EOS est ajouté, explicitement, page 10.

Construire les fenêtres glissantes

Le tokenizer produit les identifiants ; le dataset construit les exemples d'apprentissage.

Pour une fenêtre d'entrée de longueur **L**, il faut **L + 1** identifiants disponibles. La cible est décalée d'un token. Le pas, appelé `stride`, règle l'avancement de la fenêtre.

EXEMPLE NUMÉRIQUE ILLUSTRATIF

```
Identifiants : [10, 20, 30, 40, 50, 60]
L = 3 ; stride = 2

Fenetre 1 : X = [10, 20, 30]  Y = [20, 30, 40]
Fenetre 2 : X = [30, 40, 50]  Y = [40, 50, 60]
```

PYTHON

```python
def windows(ids, length=8, stride=4):
    if length < 1 or stride < 1:
        raise ValueError("length et stride doivent etre positifs.")
    for start in range(0, len(ids) - length, stride):
        x = ids[start:start + length]
        y = ids[start + 1:start + length + 1]
        yield x, y

def document_windows(tokenizer, texts, length=8, stride=4):
    eos = tokenizer.token_to_id("[EOS]")
    for text in texts:
        ids = tokenizer.encode(text, add_special_tokens=False).ids + [eos]
        yield from windows(ids, length=length, stride=stride)
```

On traite chaque document séparément et on ajoute explicitement `[EOS]`. Les fenêtres ne traversent donc pas les frontières entre documents. Si un document est trop court, il ne produit aucune fenêtre de taille fixe.

Choix	Effet
stride = 1	Chevauchement important ; beaucoup d'exemples très proches.
stride = L	Blocs d'entrée successifs sans chevauchement ; la cible reste toujours décalée d'un token.

> Séparer entraînement et validation **avant** la création des fenêtres. Sinon, des fenêtres presque identiques d'un même document peuvent se retrouver dans les deux partitions.

Exercices : à toi de construire

Réponds avant de consulter la page suivante. Les exemples sont pédagogiques.

01 / Identifiants et inconnus

Un vocabulaire contient [UNK] = 0, salam = 1 et labas = 2. Encode `salam khouya`. Peut-on retrouver khouya au décodage ?

02 / Première fusion BPE

Le corpus contient deux fois abab et une fois ab. Avec des caractères comme unités initiales, compte (a,b) et (b,a), puis effectue la première fusion.

03 / Risque de fuite

Tu apprends les fusions sur tous les documents, puis tu réserves 10 % à la validation. Quelle étape faut-il déplacer ? Et pourquoi ne pas découper directement les fenêtres au hasard ?

04 / Décodage et Unicode

L'entrée `cafe\u0301` contient cinq points de code. Le tokenizer normalise en NFC. Quel texte doit restituer decode(encode(entrée)) ? La longueur doit-elle rester cinq ?

05 / Deux fenêtres

À partir de [10,20,30,40,50,60], écris les entrées et les cibles pour L = 3 et stride = 2. Puis indique ce qui se passe si la séquence ne contient que trois identifiants, EOS déjà compris.

06 / Expérience sur AtlasIA

Apprends trois tokenizers avec des tailles cibles de 500, 1 000 et 2 000, en gardant les mêmes partitions. Relève leur taille réelle et les tokens pour 100 caractères sur la validation. Quel compromis observes-tu ?

> Pour chaque réponse, explique le mécanisme en une phrase. Le but est de pouvoir prévoir le résultat avant d'exécuter Python.

Corrigés et points de contrôle

Vérifier le raisonnement, puis relier le tokenizer au futur modèle.

01 / Une information perdue

Le résultat est [1,0]. [UNK] signale une unité inconnue, sans mémoriser sa graphie. On ne peut donc pas reconstruire khouya.

02 / La paire la plus fréquente

(a,b) apparaît cinq fois et (b,a) deux fois. Après la fusion a+b, les séquences deviennent [ab,ab], [ab,ab] et [ab].

03 / Séparer les documents d'abord

On réserve la validation avant d'apprendre le vocabulaire et les fusions. Les fenêtres d'un même document restent dans une seule partition ; les doublons ou textes apparentés doivent aussi être regroupés.

04 / Le texte normalisé

On attend café en NFC, soit quatre points de code dans cet exemple. Le tokenizer peut être réversible par rapport à sa version normalisée sans restituer la représentation Unicode originale.

05 / Le décalage obligatoire

Première entrée [10,20,30], cible [20,30,40]. Deuxième entrée [30,40,50], cible [40,50,60]. Avec trois identifiants, EOS déjà compris, et L=3, aucune fenêtre complète : il manque le quatrième identifiant pour la dernière cible.

06 / Une mesure, pas une promesse

Les valeurs dépendent des textes et des fusions apprises. Un vocabulaire plus grand peut diminuer la fragmentation, avec une table d'embeddings plus grande. On choisit après observation ; aucune valeur chiffrée n'est inventée ici.

> **Lien avec la suite :** si V est la taille réelle du vocabulaire et D la dimension choisie, la table d'embeddings contient V × D nombres. Le tokenizer fournit les indices ; le modèle apprend les vecteurs et les relations de contexte.

Lancer l'atelier et poursuivre

Un script compagnon exécute la chaîne complète, du CSV au tokenizer sauvegardé.

Enregistre `atelier_tokenizer.py` dans ton dossier de travail. Python 3.10 ou plus récent suffit pour le script. Le premier chargement du CSV public nécessite une connexion Internet ; un CSV local peut ensuite être réutilisé.

COMMANDES À EXÉCUTER DANS LE DOSSIER DU SCRIPT

```
python -m pip install tokenizers==0.22.1
python atelier_tokenizer.py --limit 1000 --vocab-size 2000

# Si tu as deja telecharge le CSV AtlasIA :
python atelier_tokenizer.py --csv atlasia_stories.csv --limit 1000
```

Le script télécharge uniquement le CSV public de la rubrique stories, puis lit la colonne darija. Il produit dans `atelier_darija/` : le tokenizer JSON et un manifeste de provenance et de mesures. Il vérifie le décodage et affiche une paire entrée/cible.

> **Si le téléchargement échoue :** ouvre le dataset [1], onglet Files, puis télécharge atlasia_stories.csv. Relance ensuite la commande avec --csv. Atlaset est un autre corpus : ses fichiers demandent une acceptation des conditions sur Hugging Face.

Sources officielles et ressources

[1] AtlasIA - dataset darija_english et licence affichée

[2] Hugging Face - ByteLevel et alphabet de 256 octets

[3] Hugging Face - BpeTrainer, paramètres d'apprentissage

[4] Hugging Face - encode, decode et sauvegarde du tokenizer

[5] Hugging Face - apprentissage depuis un itérateur

[6] Sebastian Raschka - code du chapitre 2

[7] Python - normalisation Unicode

Les références API versionnées servent à expliquer les composants ; les exemples pratiques ont été vérifiés avec Tokenizers 0.22.1. Les exemples courts inventés sont signalés ; ils ne constituent pas le corpus AtlasIA.

Vérification : fonctions testées localement sur des exemples pédagogiques, y compris les cas Unicode et les fenêtres. Le téléchargement du CSV n'a pas pu être exécuté depuis cet environnement ; le chargement local avec --csv est prévu.